

Chunking Pipeline — Design Discussion and Decisions

Date: 2026-05-23

Scope: Records the design rationale, bugs found, and decisions made while building the chunking pipeline.

This is a team reference document, not a specification. For the specification, read [CHUNKING_LOGIC.md](#).

What the pipeline does (one paragraph)

Every parsed markdown file in `parsed/` is split into self-contained chunks by `ingestion/chunkers/run.py`. Each chunk carries a context header, the body text, and structured metadata (source, evidence grade, retrieval tier, condition trigger, Kerala food flag, safety flag, etc.). Chunks are written to `data/chunks/<SOURCE>.jsonl`. The next step is the embedder, which reads these JSONL files, embeds each chunk's `text` field with `BAAI/bge-large-en-v1.5`, and upserts to pgvector (Neon).

Key design decisions

1. Split on annotation boundaries, not token count

Decision: The recommendation chunker splits at `<!-- rag_metadata -->` comment boundaries — the annotations the extractors already inserted — rather than at a fixed token window.

Rationale: Token-window chunking cuts recommendation text mid-sentence without regard for clinical meaning. A chunk that starts mid-recommendation ("...therefore HbA1c should be < 7.0%") has no retrievable context. Every recommendation in RSSDI, ADA, and ICMR-STW already has a metadata boundary either at section level or at individual recommendation level (the `_annotate_evidence_grades()` extractor pass). Following those boundaries means each chunk is a self-contained clinical statement.

Trade-off acknowledged: Sources with sparse annotations (ICMR-STW has only 14 metadata comments → 10 chunks) produce coarser chunks. This is a property of the extraction quality, not the chunking strategy. Improving annotation density in the extractor is the right fix, not switching to token-windowing.

2. Context header on every chunk

Decision: Every chunk's `text` field starts with a context header. Three variants, depending on what fields are present:

```
[SOURCE YEAR — SECTION_REF: SECTION_TITLE]  ← when both section_ref and section_title present
[SOURCE YEAR — SECTION_TITLE]                 ← section_title only (most sources)
[SOURCE YEAR]                                 ← no section info
```

Rationale: A chunk like "HbA1c target < 7.0% (Grade A)" is uninterpretable in isolation — it could be from any source, any patient population, any year. The embedding of header + body together produces a vector that carries source authority and section context. Without the header, a query like "what is the HbA1c target for Kerala T2DM patients?" retrieves the body embedding alone, which ranks equally with an older or less

authoritative source.

3. Dual-chunk strategy for ICMR-NIN food table

Decision: Two chunk types per food group with Kerala foods:

- Type A — full group table (one chunk per group, batched into 30-row fragments if > 512 tokens)
- Type B — one chunk per individual Kerala food row (individual food row + column header)

Rationale: These two types serve different retrieval queries:

- "Which Kerala fish has the highest protein?" → needs the full Marine Fish table (Type A)
- "How many carbs are in karimeen?" → needs the single karimeen row (Type B); a full-group chunk will bury the karimeen row in 100 other rows and the reranker can't promote it reliably

Both enter the same pgvector collection. The `kerala_food=true` flag distinguishes Type B chunks.

4. IDF-DAR two-pass chunking

Decision: Priority chunks (safety-redline BG thresholds, meal-timing annotations, risk-stratification blocks) are extracted first as standalone atomic chunks. Remaining text is chunked with a 2-page sliding window (1-page overlap).

Rationale: IDF-DAR has no section boundaries — it is 333 pages of raw pdfplumber text. A pure sliding window would split the safety BG thresholds ("break fast if BG < 70 mg/dL or > 300 mg/dL") across window boundaries or bury them in unrelated text. These are zero-loss safety statements: a bot that gives a wrong Ramadan BG threshold could cause a patient to maintain a dangerous fast. They must be standalone, retrievable without any surrounding context.

The meal-timing annotations (suhoor vs. iftar) are similarly separated to prevent suhoor dose-halving guidance from being retrieved in response to an iftar query.

5. Safety-critical chunks are never split

Decision: Chunks with `safety_critical=true` or `chunk_note=keep_atomic_large_window` bypass the 512-token ceiling entirely. They are emitted at full size regardless of token count.

Rationale: The 512-token ceiling exists to keep chunks retrievable and embedable. But a treatment step-up ladder (WHO HEARTS antihypertensive: Step 1 → Step 2 → Step 3) split at Step 2 is clinically dangerous — a doctor acting on the fragment starting at Step 2 has no context for when Step 2 is indicated. Same for KDIGO eGFR thresholds for Metformin dosing and IDF-DAR BG thresholds. The operational risk of an oversized chunk (slightly degraded embedding quality) is lower than the clinical risk of a split safety statement.

6. Evidence grade normalization

Decision: Five different evidence grading schemes (ADA/RSSDI A–E, KDIGO 1A–2D, ESC Class I/IIa/IIb/III × Level A/B/C, narrative consensus) are all normalized to a single stored field: `grade_priority` (integer 1–5). Raw grade strings (`evidence_grade`) and the grading scale name (`evidence_schema`) are **not stored** in the chunk metadata.

Rationale: At retrieval time, a query about "safe HbA1c targets for elderly patients" should prefer Grade A evidence from any source over Grade E from the same source. Without normalization, grade-filtered retrieval would require source-specific logic. The `grade_priority` integer collapses all schemes to a single pre-filter dimension — that is the only grade field any query ever needs.

ESC is the only source that uses two separate fields (class + level). The chunker resolves these to a `grade_priority` integer directly; neither raw field is stored. Storing only the integer was a deliberate simplification made when the 14-field metadata schema was locked — see `CHUNKING_LOGIC.md` "Fields intentionally dropped".

Bugs found and fixed during this session

Bug 1 — ICMR-NIN fish column mis-assignment (critical)

Symptom: Fish and seafood food codes (P-series, Q-series, etc.) showing physically impossible values: Karimeen with `carbohydrate_g = 386`, Anchovy with `protein_g = 77.77`.

Root cause: The IFCT PDF's fish/seafood/meat pages have different x-coordinate column layouts than the cereals section (page 41) used to calibrate the parser. The fixed x-position column detector placed `energy_kJ` values in the `carbohydrate_g` slot and moisture values in the `protein_g` slot.

Fix: Added `_correct_column_shift()` in `ingestion/parsers/food_table.py`. Detection via biological-impossibility thresholds:

- `carbohydrate_g > 100` → physically impossible for any whole food → this is actually `energy_kJ`
- `protein_g > 50` → impossible for whole fish → this is actually `moisture_%`
- When both shifted, `fiber_total_g` value (also shifted) → this is actually `fat_g`

Also added garbled-row filter: `energy_kJ > 4000` drops rows where the PDF parser concatenated values from adjacent food codes into a single number (e.g. `400 + 1083` → `4001083`).

Verified: Post-fix values for key Kerala fish: Anchovy Fat=1.62g Energy=367 kJ; Karimeen Fat=0.97g Energy=386 kJ; Mackerel Fat=1.2g Energy=423 kJ; Sardine Fat=0.84g Energy=637 kJ. These are consistent with IFCT 2017 published values.

Bug 2 — ICMR-NIN sub-table duplicates (data quality)

Symptom: Same food codes appearing 8–13 times in the parsed output. E.g. P003 (Anchovy) at Energy=367 kJ AND Energy=11.7 kJ AND Energy=7.4 kJ etc.

Root cause: The IFCT PDF has 8–13 separate nutrient sub-tables per food (proximate composition, amino acids, fatty acids fat-soluble, vitamins water-soluble, minerals, trace elements, etc.). Each sub-table creates a separate row per food code. The parser was correctly extracting all of them.

Fix: Added deduplication in `ingestion/extractors/tier1/icmr_nin.py`: keep only the highest-energy occurrence per food code. The proximate composition table (what the RAG system needs) always has the highest energy value; sub-tables for fatty acids or minerals have near-zero energy values (e.g. 7–25 kJ).

Result: 7,109 raw rows → 543 unique food codes. 6,566 sub-table rows dropped. The 543 number is consistent with IFCT 2017's documented food count.

Bug 3 — Food table chunker Kerala sub-table boundary bleed

Symptom: Karimeen (P026, Marine Fish) appearing as a Type B Kerala chunk under "Palm and Coconut Beverages" instead of "Marine Fish".

Root cause: The `kerala_sub_end` variable was set to the position of the *next* Kerala sub-heading in the file, not the next group heading (`g_end`). The next Kerala sub-heading after Palm/Coconut (K group, pos 30553) was Marine Fish (P group, pos 44471). So `kerala_text` spanned from pos 30553 to 44471 — crossing group boundaries and picking up all Marine Fish rows as if they belonged to the K group.

Fix: `kerala_sub_end = g_end` always. The group boundary is the correct ceiling. Removed the loop that was looking for the next Kerala heading position.

Bug 4 — settings.py model name mismatches

Symptom: `config/settings.py` had `embedding_model = "BAAI/bge-m3"` and `reranker_model = "BAAI/bge-reranker-v2-m3"`, both different from `base_model_spec.md` decisions (D2: BAAI/bge-large-en-v1.5; D4: BAAI/bge-reranker-large).

Fix: Updated `settings.py` to match the spec. `bge-large-en-v1.5` is the correct choice — it's a 1024-dim English model tuned for retrieval. `bge-m3` is a multilingual model that would embed Malayalam tokens differently and is not what the spec calls for at this stage.

Bug 5 — corpus_manifest.json stale parser fields

Symptom: KDIGO and IDF-DAR had `"parser": "docling"` and `"chunker": "recommendation"` in the manifest, but both actually use pdfplumber backends and `page_window` chunker.

Fix: Updated to `"parser": "pdfplumber"`, `"chunker": "page_window"` for both. Also corrected RSSDI to `"parser": "pdfplumber"` (it was correctly built as pdfplumber but manifest said docling).

IFCT actual vs. documented letter codes

The standard IFCT 2017 documentation maps food group letters differently from what the actual PDF produces. This was discovered empirically by inspecting parsed output:

Letter	Documented as	Actual (from parsed PDF)
K	(varies by source)	Palm and Coconut Beverages (K001=Palmyra palm, K002=Coconut water)
P	(varies)	Marine Fish (P001=Allathi, P026=Karimeen, P071=Sardine)
Q	(varies)	Marine Crustaceans and Shellfish
R	(varies)	Marine Mollusks
S	(varies)	Freshwater Fish and Shellfish
T	(varies)	Edible Fats and Oils

I	(varies)	Sugarcane and Sugarcane Juice
---	----------	-------------------------------

Kerala food patterns in `_KERALA_FOOD_PATTERNS` use the **actual** letters. Any future addition of Kerala food patterns must use the actual letter, not the documented one.

Why ICMR-STW has only 10 chunks

The ICMR-STW 2024 is a Standard Treatment Workflow — a condensed 4-page clinical decision guide, not a full guideline. The extractor produced 14 `rag_metadata` annotations. After filtering empty bodies, 10 chunks remain. This is correct: the STW is intentionally terse. If more granular chunking is needed, the extractor's `_annotate_algorithm_steps()` pass should be extended to annotate individual drug bullet points within each step.

Chunk counts vs. estimates

After all audits and fixes (2026-05-23), final chunk count is **4,059** with **zero chunks over 512 tokens**.

Source	Chunker	Pre-fix	Post-fix	Pre-fix max	Post-fix max	Over-512
RSSDI_2022	recommenda	270	764	2,471	512	0
ICMR_STW_2024	recommenda	10	10	251	251	0
ADA_2026	recommenda	614	658	1,995	512	0
ICMR_NIN	food_table	68	71	652	512	0
Anoop_Misra	narrative	54	58	1,741	512	0
KDIGO_2022_DM_CKD	page_windo	269	891	4,387	512	0
IDF_DAR	page_windo	462	1,103	2,501	512	0
ESC_2023_CV_DM	recommenda	153	419	2,896	512	0
WHO_HEARTS	narrative	33	20	579	512	0
Telemedicine_Guidelines_2	narrative	52	65	995	512	0
Total		1,985	4,059	4,387	512	0

Why the count grew from 1,985 to 4,059: The token ceiling was not being enforced before (Bugs 6–10).

Large annotation bodies (some up to 4,387 tokens) were emitted as single oversized chunks. After the cascade splitting and overlap fixes (Bugs 6–16), those bodies become multiple correctly-sized chunks. The quality of each chunk is higher — focused content, better embedding vectors.

Zero over-512 chunks: All chunks are at or below the 512 estimated-token ceiling. The max of exactly 512 reflects the assemble-and-measure approach (Bugs 15–16) which ensures the assembled chunk text is measured, not accumulated estimates.

Token-ceiling audit and fixes (2026-05-23)

A full audit of `data/chunks/*.jsonl` revealed that the 512-token ceiling was not being enforced correctly across multiple chunkers. Five bugs were found and fixed.

Before vs. after

Source	Max tokens before	Max tokens after	Over-512 before	Over-512 after
KDIGO_2022_DM_CKD	4,387	509	128	0
ESC_2023_CV_DM	2,896	509	85	0
IDF_DAR	2,501	509	262	0
RSSDI_2022	2,471	509	175	0
ADA_2026	1,995	509	34	0
Anoop_Misra	1,741	551	12	6
Telemedicine	995	545	26	14
ICMR_NIN	652	520	11	8
WHO_HEARTS	579	521	1	1

Bug 6 — Page-window chunker: zero token ceiling

Symptom: KDIGO chunks up to 4,387 tokens. IDF-DAR chunks up to 2,501 tokens. Spec said "2-page windows ≈ 400–600 tokens" — but KDIGO pages are far denser than the IDF-DAR baseline assumption.

Root cause: `page_window.py` had no ceiling enforcement at all. It emitted whatever 2 pages contained, regardless of size.

Fix: Added `_emit_window_chunks()` in `page_window.py`. Every window text passes through the paragraph → sentence → hard-split cascade before being emitted. Oversized windows are split into multiple chunks, each with the page range repeated in the header.

Bug 7 — Recommendation chunker: `<!-- page N -->` markers left in ESC bodies

Symptom: ESC chunks up to 2,896 tokens with `section_title=None`. These were table-of-contents pages — the `<!-- page N -->` markers left by Docling were not stripped from chunk bodies, so the entire TOC blob

became one chunk.

Root cause: The Docling extractor inserts `<!-- page N -->` markers in the markdown output. The recommendation chunker splits at `rag_metadata` boundaries but did not strip page markers from the body text between splits.

Fix: Added `_PAGE_MARKER_RE.sub("", body)` in `chunk_recommendation_source()` before body processing.

Bug 8 — All text-based chunkers: unbreakable single paragraphs

Symptom: ADA (max 1,995 tok), Anoop_Misra (1,741 tok), Telemedicine (995 tok) all had oversized chunks that the existing paragraph-split logic could not break.

Root cause: All three chunkers (`recommendation`, `narrative`, `food_table`) split at `\n\n` boundaries only. If a single paragraph was itself > 512 tokens, no split could fire and the whole paragraph was emitted as one chunk.

Fix: Added `split_text_with_ceiling()`, `split_at_sentences()`, and `hard_split_text()` to `ingestion/chunkers/base.py`. The cascade:

1. Split at `\n\n` paragraph boundaries
2. For any paragraph still > `body_max`: split at `.` `[A-Z]` sentence boundaries
3. For any sentence still > `body_max` (or text with no sentence boundaries): hard-split at last space before `body_max × 4` characters; if no space, hard-cut at character boundary

The effective body budget accounts for header token cost: `body_max = 512 - header_tokens - 4`.

Bug 9 — RSSDI parsing artifact: concatenated words, no sentence boundaries

Symptom: RSSDI chunks with garbled section titles like `Detection)Andremission`, body text like `InternationalJournalofDiabetesinDevelopingCountries (October2022) 42 (Suppl1) :S1-S143`. Up to 2,471 tokens in a single chunk.

Root cause: pdfplumber's text extraction for some RSSDI pages (particularly journal header/footer lines and dense reference sections) concatenates words without spaces. This means no `\n\n` paragraph boundaries and no `.` `[A-Z]` sentence boundaries exist for the splitting logic to use.

Fix: `hard_split_text()` handles this as the final fallback: it splits at the last space before `body_max × 4` characters, and if no space is found at all (pure concatenated run), it cuts at the exact character position. This produces semantically imperfect but correctly-sized chunks. Improving the RSSDI extractor's space normalization pass would produce cleaner splits — deferred.

Bug 10 — Food table chunker: or len(rows) <= batch_size logic bypass

Symptom: ICMR-NIN chunks up to 652 tokens despite the batching logic.

Root cause: `_emit_table_chunks()` had: `if token_estimate(full_text) <= _MAX_TOKENS or len(rows) <= batch_size: emit as single chunk`. The `or len(rows) <= batch_size` meant that any table with ≤ 30 rows was always emitted as one chunk, even if it exceeded 512 tokens.

Fix: Changed to `if token_estimate(full_text) <= _MAX_TOKENS: emit as single chunk`. For tables over 512 tokens, replaced the fixed-30-row batching with token-aware batching: accumulate rows greedily until the next row would push the chunk over 512 tokens (hard cap of 30 rows per batch also retained as a safeguard).

Overlap and token-ceiling audit (2026-05-23, second pass)

A follow-up audit checked two specific aspects in detail:

1. Is the overlap implementation correct? (narrative chunker only)
2. Is the token ceiling enforced in all edge cases? (all chunkers)

Six more bugs were found and fixed. After all fixes: 4,059 chunks, zero over 512, max exactly 512.

Bug 11 — Narrative chunker: `_last_n_tokens()` counted words, not tokens

File: `ingestion/chunkers/narrative.py` lines 24–26

Symptom: Overlap tails ~30% larger than the reserved 50-token budget. Chunks that should be ≤ 512 tokens reached 545–590 tokens because the overlap tail consumed ~65 actual tokens instead of ~50.

Root cause:

```
# Buggy — counts WORDS, not tokens
def _last_n_tokens(text: str, n: int) -> str:
    words = text.split()
    return " ".join(words[-n:]) if len(words) > n else text
```

"`".join(words[-50:])`" for typical clinical prose returns ~325 chars = ~81 estimated tokens, not 50. The word-count and token-count approximations diverge because medical terms like "HbA1c", "metformin", "mg/dL" are single words but can be 3–5 characters each.

Fix:

```
def _last_n_tokens(text: str, n: int) -> str:
    max_chars = n * 4          # 50 tokens * 4 chars/token = 200 chars
    if len(text) <= max_chars:
        return text
    start_pos = max(0, len(text) - max_chars)
    while start_pos < len(text) and text[start_pos] not in (" ", "\n"):
        start_pos += 1          # walk to next word boundary
    return text[start_pos:].rstrip()
```

Character-budget approach ($n \times 4$) matches how `token_estimate()` works throughout the codebase. Walking forward to the next word boundary ensures we never cut mid-word; walking forward (not backward) means the result is always $\leq \text{max_chars}$.

Bug 12 — Narrative chunker: single `body_max` reserved overlap budget on first chunk

File: `ingestion/chunkers/narrative.py` line 40

Symptom: First fragment of each split narrative section wasted ~50 tokens of available space. No incorrect output (chunks were under 512), but inefficient — the first chunk carried only ~440 tokens of content when it could carry ~490.

Root cause: A single `body_max` was computed as `512 - header_cost - 50 - 4`, reserving 50 tokens for the overlap tail even for the first chunk which has no overlap prepended.

Fix: Two separate budgets:

```
body_max_first = max(64, _MAX_TOKENS - header_cost - 4)
body_max_rest  = max(64, _MAX_TOKENS - header_cost - _OVERLAP_TOKENS - 4)
```

The first chunk uses the larger budget; all subsequent chunks use the tighter budget.

Bug 13 — Narrative chunker: last-fragment `make_chunk_id` used `section_ref` instead of `section_title`

File: `ingestion/chunkers/narrative.py` line 83

Symptom: Last fragment of a multi-fragment narrative section generated a different `chunk_id` hash than its earlier siblings. If a section had no `section_ref`, the last fragment used `"nosec"` in its fingerprint while siblings used `section_title`. This caused silent hash collisions between last-fragment IDs across different sections with missing refs.

Root cause:

```
# Buggy — uses section_ref for last fragment, section_title for all others
c.chunk_id = make_chunk_id(base.source, base.section_ref, c.text)
```

All other `make_chunk_id` calls in the narrative chunker correctly use `base.section_title`. This was a copy-paste oversight.

Fix: `make_chunk_id(base.source, base.section_title, c.text)` on the last fragment, consistent with all other calls.

Bug 14 — `hard_split_text()` could append empty fragment when `rfind` returns 0

File: `ingestion/chunkers/base.py` lines 138–142

Symptom: Rare edge case: if the first character in a text chunk was a space (e.g. a leading-space paragraph), `rfind(" ", 0, max_chars)` could return 0. `text[:0].strip() = ""`. An empty string would be appended to `fragments` and later emit as a zero-content chunk.

Root cause:

```
split_at = text.rfind(" ", 0, max_chars)
if split_at == -1:          # only catches "no space found"
    split_at = max_chars
fragments.append(text[:split_at].strip()) # empty if split_at == 0
```

Fix:

```
if split_at <= 0:          # catches both "no space" and "space at start"
    split_at = max_chars
fragment = text[:split_at].strip()
if fragment:              # guard against empty string
    fragments.append(fragment)
```

Bug 15 — Narrative chunker: pre-expansion used first-chunk budget, not rest-chunk budget

File: `ingestion/chunkers/narrative.py` (pre-expansion loop)

Symptom: After fixing Bug 12 (two-budget design), chunks still reached 557 tokens. Narrative sections with large paragraphs expanded to fit `body_max_first` (490 tokens) could later be placed in a rest-chunk position where the 50-token overlap tail is also prepended, giving: 18 (header) + 50 (overlap) + 490 (body) = 558 tokens.

Root cause: The pre-expansion step used `body_max_first` to decide whether to split a paragraph:

```
for para in paragraphs:
    if token_estimate(para) > body_max_first:    # 490 — too large
        expanded.extend(split_at_sentences(para, body_max_first))
```

A paragraph expanded to exactly 490 tokens is safe for the first chunk (no overlap). But if there are earlier chunks, this same paragraph lands in a rest-chunk position with overlap prepended and becomes 490 + 50 + 18 = 558 tokens.

Fix: Always expand using `body_max_rest` (440 tokens):

```
for para in paragraphs:
    if token_estimate(para) > body_max_rest:    # 440 — tight budget always safe
        expanded.extend(split_at_sentences(para, body_max_rest))
```

The first chunk loses ~50 tokens of headroom (unused since no overlap is prepended). This is the correct trade-off: predictable safety over maximum packing of the first chunk.

Bug 16 — Accumulated token estimates undercount due to floor-division rounding

Files: `ingestion/chunkers/narrative.py` (accumulation loop), `ingestion/chunkers/food_table.py` (row batching)

Symptom: After fixing Bugs 11–15, chunks still reached 520 tokens. All remaining over-512 chunks were from ICMR-NIN (max 520, food table) and Telemedicine (max 515, narrative). The overruns were small but consistent (7–8 tokens).

Root cause: Both chunkers tracked an accumulated token count by summing `token_estimate()` calls on individual fragments:

```
# Undercount accumulation:
current_tok += token_estimate(row + "\n") # floor(len // 4) per row
# ...later, assembled text:
f"{header}\n\n{table_header}\n" + "\n".join(rows) # joined text estimate is higher
```

`token_estimate = len(text) // 4` uses integer floor division. Summing floor divisions is always $\leq$ the floor division of the sum:

- $3 // 4 + 3 // 4 = 0 + 0 = 0$ but $(3+3) // 4 = 1$
- Each `"\n"` and `"\n\n"` separator adds 1–2 unaccounted chars
- For 30 table rows, up to 8 tokens of systematic undercount

Fix — assemble-and-measure in both chunkers:

Narrative (`_split_with_overlap`):

```
# Assemble full candidate text, measure once:
candidate_text = _assemble(current_paras + [para], overlap_tail if chunks else "")
if token_estimate(candidate_text) > _MAX_TOKENS and current_paras:
    flush()
```

Food table (`_emit_table_chunks`):

```
def _candidate_text(batch, extra_row):
    tbl = f"{table_header_lines}\n" + "\n".join(batch + [extra_row])
    return f"{header}\n\n{tbl}"

if token_estimate(_candidate_text(current_batch, row)) > _MAX_TOKENS:
    flush()
```

Measuring the assembled text avoids all rounding issues. The check is slightly more expensive (string concatenation per candidate), but the chunker runs offline and correctness matters more than speed.

Result after this fix: Max `token_estimate` = 512, zero over-512 chunks.

Grade normalization audit and fixes (2026-05-23)

Four bugs were found in `ingestion/chunkers/base.py` during a focused audit of the evidence grade normalization logic. All fixed in the same session.

Bug 17a — KDIGO 2A and 1B both mapped to priority 2

File: `ingestion/chunkers/base.py` — `EVIDENCE_NORMALIZATION` dict

Symptom: KDIGO "1B" (strong recommendation, moderate evidence) and "2A" (weak recommendation, high evidence) both produced `grade_priority = 2`. A retrieval filter of `grade_priority <= 2` would include weak-recommendation content at the same rank as strong-recommendation content.

Root cause: The priority mapping treated evidence quality (the letter) as the sole determinant. The first digit of KDIGO grades encodes recommendation *strength*

(1 = "We recommend" / strong; 2 = "We suggest" / weak) — a clinically meaningful axis that was being collapsed. For a patient-facing system, a strong recommendation with moderate evidence (1B) should outrank a weak recommendation with high evidence (2A).

Fix: "2A" → `grade_priority = 3` (was 2). The full corrected mapping:

Grade	Strength	Evidence quality	Priority (old)	Priority (new)
1A	strong	high	1	1 (unchanged)
1B	strong	moderate	2	2 (unchanged)
1C	strong	low	3	3 (unchanged)
1D	strong	very low	4	4 (unchanged)
2A	weak	high	2	3 ← fixed
2B	weak	moderate	3	3 (unchanged)
2C	weak	low	4	4 (unchanged)
2D	weak	very low	5	5 (unchanged)

Bug 17b — ESC Class-III treated as low priority, not as a contraindication signal

File: `ingestion/chunkers/base.py` — `normalize_esc_grade()`

Symptom: ESC `Class-III` recommendations ("intervention not recommended / potentially harmful") were assigned `grade_priority = 4` or `5` — treated identically to "low-quality evidence". They were not flagged as safety-critical and could be excluded from retrieval on safety queries (which filter `grade_priority <= 2`).

Root cause: `class_priority = {"I": 1, "IIa": 2, "IIb": 3, "III": 4}` treated Class-III as the bottom of a quality scale. ESC Class-III does not mean "poor evidence" — it means "do not do this / this causes harm". The bot must retrieve these chunks to know what NOT to advise. Missing a contraindication signal is a patient safety risk.

Fix:

- `normalize_esc_grade()` now special-cases "III": returns (merged, "ESC_Class_III_HARM", 5).
- The recommendation chunker detects `ev_schema == ESC_CLASS_III_HARM` and sets `safety_critical = True` — ensuring these chunks are always retrieved on relevant queries.
- The `ESC_CLASS_III_HARM` sentinel constant is exported from `base.py` and imported in `recommendation.py`.

Bug 17c — ESC class/level lookup broke on uppercase input

File: `ingestion/chunkers/base.py` — `normalize_esc_grade()`

Symptom: When the table scanner in `recommendation.py` passed `cls_val.upper()` (e.g. "IIA", "IIB") to `normalize_esc_grade()`, the dict lookup failed silently and returned priority 5. Only the inline annotation path (which passes mixed-case "IIa", "IIb") worked correctly.

Root cause: The dict used mixed-case keys `{"IIa": 2, "IIb": 3}` but the table scanner always uppercases values before calling the function. `"IIA"` is not in the dict, so it fell through to the default of 4 (itself wrong — see Bug 17d), added `level_modifier.get(lvl, 1)`, producing priority 5.

Fix: `normalize_esc_grade()` now normalises both inputs to uppercase internally before lookup. The class dict keys are updated to `{"I": 1, "IIA": 2, "IIB": 3}` — fully case-insensitive for all callers.

Bug 17d — ESC unknown class defaulted to 4, not 5

File: `ingestion/chunkers/base.py` — `normalize_esc_grade()`

Symptom: Any ESC `evidence_class` value not in the dict (e.g. unrecognised annotation like `"Class IIc"`) returned `class_priority.get(cls, 4) = 4`, then added a level modifier, producing priority 4 or 5. Priority 4 is higher than the "unknown/ungraded" floor of 5 used everywhere else in the normalization logic — inconsistent.

Fix: Default changed from 4 → 5: `class_priority.get(cls, 5)`. All unknown inputs now consistently produce priority 5. The existing warning log in `normalize_grade()` surfaces the gap.

Bug 17e — `normalize_grade()` silently discarded unrecognised grade strings

File: `ingestion/chunkers/base.py` — `normalize_grade()`

Symptom: Grade strings not in `EVIDENCE_NORMALIZATION` (e.g. `"Strong"`, `"Moderate"`, `"Level B"`) returned `(raw, None, 5)` with no log output. Extractor annotation gaps during corpus development were invisible until a manual audit.

Fix: Added `_logger.warning(...)` for unrecognised grade strings. The pipeline continues (priority 5 assigned) but the gap appears in logs during corpus development runs.

What is NOT done yet

1. Embedder + pgvector upsert — the next engineering step; reads `data/chunks/*.jsonl`, embeds with `bge-large-en-v1.5`, upserts to pgvector (Neon). Full design in `CLAUDE.md` → Embedder section.
2. Deduplication at upsert — `text_hash` field is in every chunk JSON; the upsert pipeline checks for duplicates before inserting (schema in `CHUNKING_LOGIC.md`)
3. Retrieval pipeline — query → embed → pgvector ANN → rerank → LLM
4. KDIGO annotation passes — KDIGO has no `_annotate_*` passes yet; raw page text is chunked as page windows; adding `eGFR-threshold` and `recommendation-block` annotations to the extractor would produce semantically cleaner chunks (right now KDIGO chunks are correctly sized but carry no metadata beyond page range)
5. RSSDI extractor space normalization — `pdfplumber` concatenates words on some RSSDI pages

(journal header/footer lines, dense reference sections); adding a `re.sub(r'([a-z])([A-Z])', r'\1 \2, text)` pass in the RSSDI extractor would fix the root cause; current `hard_split_text()` fallback handles it acceptably

6. Lay-term bridging in `topic_tags` — "kappa" should also tag "tapioca", "mathi" → "sardine", "ayala" → "mackerel", "chaaya" → "sweet tea"; deferred
7. `medication_mentions` metadata field — a regex pass over chunk body to extract drug names into a queryable field; deferred